

MOSAICO

Visión general de las siete pestañas

Arquitectura funcional completa de Mosaico, límites entre módulos y flujo de trabajo

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Identidad y alcance del producto
02	Las siete pestañas
03	Flujo principal de trabajo
04	Modelo compartido de proyecto
05	Límites entre pestañas
06	Sistemas transversales
07	Arquitectura técnica recomendada
08	Prioridades y exclusiones
09	Criterios globales
10	Fuentes conceptuales

1. Identidad y alcance del producto

Mosaico es un entorno de producción 2D centrado en pixel art y desarrollo de videojuegos. Su objetivo no es reemplazar un motor completo, sino unir en una sola aplicación el dibujo manual, la preparación de recursos, la composición procedural, la construcción manual y reglada de mapas, la organización de mundos, el procesamiento por lotes y la exportación hacia motores.

Decisión de producto

Mosaico debe sentirse como una cadena de producción integrada. Cada pestaña resuelve una escala distinta del mismo proyecto y comparte archivos, IDs, paletas, tilesets, reglas, animaciones y metadatos.

- Pixel Art: edición manual de imágenes, sprites y animaciones.
- Assets: biblioteca, preparación, slicing, atlas y metadatos.
- Pipelines: composición procedural y VFX mediante nodos no destructivos, inspirada en Pixel Composer.
- Mapas: niveles basados en tiles, con edición manual y generación pattern-based mediante reglas inspirada en TileKit.
- Mundo: organización macro de múltiples mapas, regiones y conexiones.
- Jobs: cola de tareas pesadas, repetibles o por lotes.
- Exportar: construcción de paquetes para Godot, Unity, Unreal y Morph.

2. Las siete pestañas

Pestaña	Pregunta que responde	Resultado principal
Pixel Art	¿Cómo se dibuja o anima este recurso?	Documento raster editable, frames, capas y spritesheet.
Assets	¿Cómo se prepara, clasifica y reutiliza?	Asset normalizado con metadatos, previews, tags y variantes.
Pipelines	¿Cómo se genera o transforma una imagen de forma procedural?	Grafo de nodos reutilizable y resultados visuales no destructivos.
Mapas	¿Cómo se construye o genera un nivel?	Mapa manual o mapa de entrada + reglas + resultado derivado, con capas y objetos.
Mundo	¿Cómo se conectan y organizan varios niveles?	Grafo mundial, regiones, portales, coordenadas y reglas globales.
Jobs	¿Qué procesos están corriendo o deben repetirse?	Ejecuciones con progreso, logs, caché, reintentos y resultados.
Exportar	¿Cómo se lleva el proyecto a un motor?	Paquete validado y adaptado a Godot, Unity, Unreal o Morph.

3. Flujo principal de trabajo

1. Crear o importar arte en Pixel Art o Assets.
2. Normalizar el recurso en Assets: tamaño, pivote, slicing, tags y tipo.
3. Aplicar un Pipeline cuando se necesiten efectos, variaciones, patrones visuales, animación procedural o composición no destructiva.
4. Construir un nivel en Mapas de forma manual o pintando un mapa básico de entrada que un ruleset transforma en tiles finales.
5. Corregir el resultado mediante capas manuales de override o convertirlo en una versión horneada.
6. Conectar varios mapas y definir regiones en Mundo.
7. Ejecutar tareas pesadas o masivas desde Jobs.
8. Validar y exportar el contenido desde Exportar al motor elegido.

```
Pixel Art / Assets -> Pipelines -> Assets -> Mapas (Manual + Reglas) -> Mundo
                        |
                        +-----> Jobs <-----+
                        |
                        Exportar
```

Flujos alternativos

No todo recurso debe pasar por todas las pestañas. Un icono puede ir de Pixel Art directamente a Exportar; una lluvia procedural puede nacer en Pipelines y colocarse en un mapa; un tileset importado puede ir de Assets a Mapas; un mapa puede combinar base reglada y ajustes manuales.

4. Modelo compartido de proyecto

Todas las pestañas deben operar sobre un único proyecto versionado. El formato interno debe separar archivos fuente, resultados derivados, metadatos y caché para evitar duplicaciones y permitir reconstruir resultados.

Entidad	Contenido mínimo
Project	ID, versión de schema, ajustes, paleta predeterminada, unidades, carpetas y perfiles de exportación.
Asset	ID estable, tipo, fuente, dimensiones, pivote, tags, licencia, dependencias y revisiones.
Graph	Nodos, conexiones, parámetros, entradas, salidas, versión y dependencias.
Ruleset	Paleta de entradas, patrones, condiciones, salidas, prioridades, transformaciones, seed y versión.
Map	Grilla, capas fuente, capas derivadas, overrides, tilesets, rulesets, objetos, colisiones y propiedades.
World	Regiones, mapas, coordenadas, conexiones, reglas globales y grupos de streaming.
Job	Tipo, entrada, configuración, estado, progreso, log, resultado, hash y tiempo.
Export Profile	Motor, versión, rutas, escalado, convenciones, opciones y validadores.

```
project.mosaico/  
  project.json  
  sources/  
  assets/  
  graphs/  
  rulesets/  
  maps/  
  worlds/  
  jobs/  
  export_profiles/  
  cache/  
  builds/
```

5. Límites entre pestañas

Confusión posible	Regla de separación
Pixel Art vs Pipelines	Pixel Art modifica manualmente píxeles y frames. Pipelines construye operaciones visuales conectadas y regenerables.
Pipelines vs reglas de Mapas	Pipelines procesa imágenes/VFX. Las reglas de Mapas transforman una grilla semántica o básica en tiles, objetos y capas espaciales.
Assets vs Pixel Art	Assets administra y prepara. Pixel Art edita el contenido visual.
Mapas vs Mundo	Mapas define un nivel. Mundo define la relación entre muchos niveles.
Mapas vs Jobs	Mapas define reglas y solicita regeneración. Jobs ejecuta regeneraciones grandes sin bloquear la interfaz.
Pipelines vs Jobs	Pipelines describe qué hacer. Jobs ejecuta, agenda, repite y registra la ejecución.
Mapas vs Exportar	Mapas crea datos neutrales y resultados horneados. Exportar los adapta a un motor concreto.

Confusión posible	Regla de separación
Mundo vs motor	Mundo describe estructura y metadatos, pero no reemplaza la lógica de gameplay ni el runtime del motor.

6. Sistemas transversales

- Undo/redo por documento, con operaciones agrupadas y memoria limitada configurable.
- Guardado automático, recuperación tras cierre inesperado y archivos de respaldo rotativos.
- IDs estables para impedir que una exportación o renombrado rompa referencias.
- Buscador global, favoritos, tags y navegación “mostrar en pestaña”.
- Sistema de notificaciones no invasivo para errores, advertencias y trabajos terminados.
- Atajos configurables y patrones de interacción consistentes en las siete pestañas.
- Preview pixel-perfect, zoom entero, opción de cuadrícula y prohibición de antialiasing involuntario.
- Seeds explícitas para reproducir variaciones aleatorias de reglas y Pipelines.
- Soporte CPU como base; GPU opcional sin depender de CUDA.

7. Arquitectura técnica recomendada

Capa	Responsabilidad
Core de proyecto	Persistencia, IDs, dependencias, migraciones y eventos.
Core raster	Capas, frames, paletas, selección, transformaciones y composición.
Core procedural	Grafo tipado, evaluación incremental, caché, tiempo y simulación.
Rule engine	Matching de patrones, prioridades, multi-pass, seeds, diagnósticos y actualización incremental.
Core tile/world	Tilesets, mapas, capas fuente/resultado, objetos, regiones, conexiones y coordenadas.
Job runner	Cola, workers, cancelación, logs, prioridades y resultados.
Export adapters	Conversión neutral -> Godot/Unity/Unreal/Morph.
UI shell	Pestañas, docks, comandos, historial, paneles y preferencias.

8. Prioridades y exclusiones

Primero cerrar el flujo vertical

Antes de añadir cientos de nodos o exportadores perfectos, Mosaico debe completar un flujo real: crear asset -> preparar tileset -> pintar mapa básico -> aplicar reglas -> corregir -> exportar.

- No desarrollar un motor de juego completo dentro de Mosaico.
- No duplicar el motor de reglas dentro de Pipelines.
- No reemplazar el modo manual: reglas y edición directa deben convivir.
- No implementar simulaciones avanzadas antes de que el grafo básico sea estable.
- No crear adaptadores de exportación que escriban directamente sobre proyectos sin preview, diff y copia de seguridad.
- No depender de servicios en la nube para funciones esenciales.

9. Criterios globales

- [] Un proyecto puede abrirse y cerrarse sin perder referencias entre las siete pestañas.
- [] Cada resultado derivado registra su fuente y puede regenerarse.
- [] Las reglas de mapa producen el mismo resultado con las mismas entradas, versión y seed.
- [] Los errores de una pestaña no corrompen el proyecto completo.
- [] Los procesos largos son cancelables y no congelan la interfaz.

- [] Los cambios se muestran correctamente a zoom entero y sin suavizado accidental.
- [] Los formatos internos tienen versión y migraciones probadas.
- [] Los exportadores generan un manifiesto legible y una lista de advertencias.
- [] La UI mantiene terminología y comportamiento consistente.

10. Fuentes conceptuales

Pipelines toma como referencia funcional a Pixel Composer, un compositor VFX basado en nodos para pixel art. Mapas toma como referencia específica a TileKit, que centra su flujo en un mapa de entrada de tiles básicos y reglas definidas por el usuario para producir un mapa de salida a partir de un tileset. Mosaico debe adoptar esos principios sin copiar interfaces ni limitarse al alcance exacto de esas herramientas.

- Pixel Composer - sitio y documentación oficiales: pixel-composer.com y docs.pixel-composer.com.
- TileKit by rxi - página oficial en [itch.io](https://rxi.itch.io/tilekit): rxi.itch.io/tilekit.
- TileKit se describe oficialmente como un tilemap editor centrado en pattern-based auto-tiling y en un sistema único de reglas por patrones.
- Fuentes consultadas el 30/07/2026.

MOSAICO

Pestaña Pixel Art

Editor raster manual, animación por frames y núcleo visual de Mosaico

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito
02	Tipos de documento
03	Interfaz
04	Herramientas
05	Capas y selecciones
06	Animación
07	Paletas
08	Importación y exportación interna
09	Integraciones
10	Criterios de aceptación

1. Propósito

Pixel Art es la pestaña de autoría manual. Debe permitir crear, corregir y animar sprites con control directo sobre cada píxel. Es el núcleo visual del producto y establece los patrones de interacción que deben imitar las demás pestañas.

Alcance

Esta pestaña no debe convertirse en un compositor procedural. Las transformaciones regenerables, simulaciones y cadenas de efectos pertenecen a Pipelines.

2. Tipos de documento

Tipo	Uso	Datos
Imagen	Iconos, retratos, fondos, tiles y objetos.	Lienzo, capas, paleta, guías y metadatos.
Animación	Personajes, VFX, UI animada.	Frames, duración, etiquetas, capas y onion skin.
Tileset	Conjunto de tiles editables.	Grilla, celdas, reglas opcionales y nombres.
Spritesheet	Importación o edición de hojas.	Slicing, secuencias, pivotes y tags.

3. Interfaz

- Barra superior: nuevo, abrir, guardar, deshacer, rehacer, tamaño, zoom y exportación rápida.
- Barra lateral izquierda: herramientas de dibujo, selección y transformación.
- Centro: canvas con zoom entero, pan, cuadrícula y guías.
- Lateral derecho: color, paleta, capas, propiedades y referencias.
- Zona inferior: timeline cuando el documento tiene animación.
- Estado contextual: coordenada, índice de color, tamaño de selección y memoria.

4. Herramientas

Grupo	Herramientas mínimas
Dibujo	Lápiz de tamaño variable, línea, rectángulo, círculo, polígono, balde y reemplazo de color.
Borrado	Borrador, borrar color global, limpiar selección y transparencia protegida.
Selección	Rectangular, elíptica, lazo, varita, por color y por capa.
Transformación	Mover, voltear, rotar 90°, escalar con vecino más cercano, recortar y anclar.
Muestreo	Cuentagotas, inspección de índice y captura de paleta.
Asistencia	Simetría, tiled preview, guías, referencias y dithering manual.

5. Capas y selecciones

- Capas visibles, ocultas, bloqueadas, agrupadas y con opacidad.
- Modos de mezcla limitados inicialmente a los que puedan mantenerse pixel-perfect.
- Máscaras de capa como fase posterior, con preview claro de su efecto.
- Selecciones flotantes con confirmación o cancelación explícita.
- No dibujar sobre una capa oculta o bloqueada; mostrar explicación inmediata.

6. Animación

- Timeline por frames con duración individual y reproducción a FPS configurables.
- Onion skin anterior y posterior, rango y opacidad ajustables.
- Etiquetas de animación: idle, walk, attack, hit, death u otras libres.
- Duplicar, insertar, borrar y reordenar frames.
- Cels enlazados opcionales para reutilizar contenido sin duplicarlo.
- Exportación interna como secuencia, GIF de preview o spritesheet.

7. Paletas

- Paleta local y biblioteca de paletas del proyecto.
- Colores indexados opcionales, rampas, bloqueo y orden manual.
- Reemplazo global de color con preview.
- Detección de colores casi duplicados y advertencia, no corrección automática destructiva.
- Importación GPL, HEX/CSV y extracción desde imagen.

8. Importación y exportación interna

Entrada	Comportamiento
PNG/JPG/WebP	Importar como imagen o referencia; advertir pérdida de transparencia cuando corresponda.
GIF/APNG	Importar frames y tiempos si el formato lo permite.
Aseprite	Preservar capas, frames y tags cuando sea técnicamente viable.
Spritesheet	Abrir asistente de slicing con preview.

Las salidas para otras pestañas no deben ser simples copias. Deben registrar una referencia al documento fuente, la revisión usada y los parámetros de preparación.

9. Integraciones

- Enviar a Assets como sprite, tile, icono, fondo, retrato o VFX.
- Abrir en Pipelines como entrada viva o snapshot congelado.
- Usar como tileset en Mapas mediante el asistente de Assets.
- Crear Job para exportar múltiples animaciones o variantes.
- Incluir en perfiles de Exportar sin perder el vínculo con el original.

10. Criterios de aceptación

- [] Todas las herramientas dibujan en coordenadas enteras.
- [] El zoom no introduce suavizado.
- [] Undo/redo cubre dibujo, capas, frames y transformaciones.
- [] Los archivos se recuperan tras un cierre inesperado.
- [] La exportación mantiene transparencia y orden de frames.
- [] Los atajos pueden configurarse.
- [] Un documento puede enviarse a Assets y reabrirse desde su referencia.

MOSAICO

Pestaña Assets

Biblioteca central, preparación y normalización de recursos del proyecto

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito
02	Modelo de asset
03	Biblioteca
04	Importación
05	Procesamiento
06	Slicing y animaciones
07	Atlas
08	Metadatos
09	Integración
10	Criterios

1. Propósito

Assets es la biblioteca central de Mosaico. Recibe archivos externos y resultados de Pixel Art o Pipelines, los clasifica y los convierte en recursos listos para Mapas, Mundo y Exportar.

Regla principal

Assets no debe editar píxel por píxel ni construir efectos complejos. Debe preparar, describir, validar y organizar.

2. Modelo de asset

Campo	Descripción
ID	Identificador estable independiente del nombre y la ruta.
Tipo	Sprite, animación, tile, tileset, icono, retrato, fondo, VFX, audio de referencia, modelo 3D de apoyo u otro.
Fuente	Archivo original, documento Pixel Art o salida de Pipeline.
Revisión	Hash y número de revisión para detectar cambios.
Dimensiones	Ancho, alto, frames, escala lógica y unidad.
Pivote	Origen local, punto de suelo o centro.
Metadatos	Tags, licencia, autor, notas, categorías y propiedades para motor.
Dependencias	Paletas, pipeline, tileset, material, animaciones u otros assets.

3. Biblioteca

- Vista de cuadrícula y lista con miniaturas generadas en caché.
- Filtros por tipo, tag, tamaño, paleta, origen, estado y fecha.
- Colecciones manuales y colecciones inteligentes por consulta.
- Favoritos, usados recientemente, sin usar y con errores.
- Panel de dependencias y usos: en qué mapas, mundos, pipelines o exports aparece.
- Renombrado seguro que no rompa IDs ni referencias.

4. Importación

- Arrastrar archivos o seleccionar carpetas completas.
- Detección de duplicados por hash y opción de vincular, reemplazar o mantener variante.
- Preview antes de confirmar; nunca modificar el original externo sin permiso.
- Presets de importación por carpeta o tipo de archivo.
- Importación en lote enviada a Jobs cuando supera un umbral.

5. Procesamiento

Operación	Opciones mínimas
Escalado	Tamaño objetivo, factor entero, nearest-neighbor, anclaje y transparencia.
Recorte	Transparencia, margen, tamaño fijo, centro o pivote.
Padding	Uniforme o por lado; color transparente o sólido.
Paleta	Mapear a paleta, limitar colores, preservar alpha y comparar antes/después.
Normalización	Nombres, rutas lógicas, pivotes, FPS y tags.

Operación	Opciones mínimas
Conversión	PNG, secuencia, spritesheet, atlas y formatos neutrales internos.

6. Slicing y animaciones

- Asistente con tamaño de celda, margen, spacing, offset y orden.
- Preview de celdas numeradas y descarte de celdas vacías.
- Agrupar frames en clips y asignar nombre, FPS, loop y dirección.
- Pivote común o por frame.
- Detectar tamaños variables y ofrecer slicing por regiones transparentes.

7. Atlas

- Empaquetado determinista con padding y extrusión configurables.
- Páginas múltiples cuando se exceda el tamaño máximo.
- Rotación desactivada por defecto para evitar problemas de pixel art.
- Manifiesto neutral con rectángulos, pivotes, nombres y clips.
- Golden preview para detectar sangrado o offsets.

8. Metadatos

Categoría	Ejemplos
Visual	Paleta, escala, dirección, punto de suelo, nueve cortes.
Gameplay neutral	Tipo de colisión sugerido, socket, categoría, daño visual, tamaño de celda.
Producción	Autor, licencia, fuente, estado, versión y notas.
Exportación	Import settings por motor, compresión, filtro, mipmaps y grupo.

9. Integración

- Editar fuente en Pixel Art.
- Abrir fuente o resultado en Pipelines.
- Usar como tileset, objeto o fondo en Mapas.
- Asignar a regiones o referencias globales en Mundo.
- Crear Jobs de conversión, atlas, validación o miniaturas.
- Añadir a un perfil de Exportar.

10. Criterios

- [] La importación nunca destruye archivos externos.
- [] Un asset conserva ID al renombrarse o moverse.
- [] Slicing y atlas son deterministas.
- [] Se puede saber dónde se usa cada asset.
- [] Los cambios de fuente marcan resultados derivados como desactualizados.
- [] Las operaciones en lote no congelan la UI.
- [] Mapas puede recibir un tileset preparado sin volver a configurarlo.

MOSAICO

Pestaña Pipelines

Composición procedural, VFX y generación no destructiva mediante nodos

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito y referencia
02	Principios
03	Interfaz del grafo
04	Sistema de datos
05	Familias de nodos
06	Animación
07	Simulación
08	3D de apoyo
09	Evaluación y caché
10	Colecciones
11	Integraciones
12	Fases
13	Criterios

1. Propósito y referencia

Pipelines es el laboratorio procedural de Mosaico. Permite conectar nodos para generar, transformar, combinar, animar y simular resultados de pixel art sin alterar los archivos fuente. El referente funcional principal es Pixel Composer, no TileKit ni un editor de reglas de mapas.

Referencia correcta

Pixel Composer se presenta oficialmente como generador, editor y compositor VFX de pixel art basado en nodos, con operaciones no destructivas, animación de parámetros, feedback, física, fluidos y funciones 3D. Mosaico debe adoptar el concepto y construirlo por fases.

2. Principios

- No destructivo: una salida puede regenerarse desde sus entradas y parámetros.
- Tipado: cada conexión conoce si transporta imagen, número, color, máscara, vector, colección, frame, geometría o simulación.
- Determinista cuando sea posible: semillas explícitas para ruido y generación.
- Incremental: solo recalcular nodos afectados.
- Inspectable: cada nodo ofrece preview y diagnóstico.
- Reusable: grupos, colecciones, presets y subgrafos.
- Animable: cualquier parámetro compatible puede usar keyframes, curvas o expresiones.

3. Interfaz del grafo

- Canvas infinito con zoom, pan, minimapa, selección múltiple y alineación.
- Biblioteca de nodos buscable por nombre, categoría, tags y compatibilidad de puerto.
- Inspector contextual para parámetros, unidades, rangos y animación.
- Panel de preview con canales RGBA, alpha, máscara, zoom entero y comparación A/B.
- Timeline y editor de curvas cuando existan parámetros animados.
- Breadcrumbs para entrar y salir de grupos o subgrafos.
- Indicadores de caché, tiempo, memoria, error y nodo desactualizado.

4. Sistema de datos

Tipo	Ejemplos
Surface	Imagen RGBA, secuencia de imágenes, tile o textura.
Mask	Escala de grises o selección binaria.
Value	Entero, float, booleano, semilla, tiempo y frame.
Vector	Vec2, Vec3, dirección, velocidad, posición y UV.
Color/Palette	Color, gradiente, rampa y paleta indexada.
Path/Shape	Línea, curva, polígono, contorno y región.
Array	Colección de superficies, valores, colores u objetos.
Simulation	Estado de partículas, rígidos, humo, fluidos o verlet.
Geometry/Scene	Malla, cámara, luz y render 3D de apoyo.

5. Familias de nodos

Familia	Nodos iniciales y futuros
Entrada/salida	Asset, archivo, color, número, tiempo, frame, preview, guardar, publicar como asset.

Familia	Nodos iniciales y futuros
Canvas y forma	Lienzo, rectángulo, círculo, línea, polígono, texto pixelado, path y máscara.
Generación	Ruido, checker, stripes, celdas, voronoi, gradiente, tiles, patrones y semillas.
Transformación	Mover, escalar, rotar 90°, flip, crop, tile, warp y nueve cortes.
Color	Paleta, cuantización, posterize, reemplazo, rampas, HSV/RGB, outline y dither.
Filtro	Blur pixel-aware, edge, sharpen discreto, distance field y morphology.
Composición	Blend, stack, mask, merge, switch, array, repeat, feedback y layer composite.
Animación	Oscilador, curve, easing, remap, delay, loop, sequence y slideshow.
Partículas/VFX	Spawner, fuerza, gravedad, vortex, trail, render de puntos y sprites.
Simulación	Rigidbody, smoke, fluid, strand y verlet en fases avanzadas.
3D	Importar malla, cámara, luz, material simple, render a baja resolución y sprite stack.
Utilidad	Debug, profiler, comment, reroute, group IO, cache y seed.

6. Animación

- Keyframes por propiedad con interpolación step como opción predeterminada para pixel art.
- Curvas, easing y expresiones matemáticas.
- Variables globales de tiempo, frame, FPS, duración y loop.
- Dopesheet y editor de curvas sincronizados con preview.
- Exportación de secuencia, spritesheet, GIF/APNG de preview o video cuando corresponda.

7. Simulación

Las simulaciones necesitan estado entre frames, caché y controles de reinicio. No deben evaluarse como nodos raster comunes.

- Dominio de simulación con resolución independiente del tamaño final.
- Seed, frame inicial, substeps, gravedad, límites y colisionadores.
- Bake a caché para reproducibilidad.
- Invalidación clara cuando cambia un parámetro anterior.
- Límites de memoria, resolución y duración para proteger equipos modestos.
- Ejecución larga delegada a Jobs.

8. 3D de apoyo

No es un modelador 3D

El 3D existe para producir sprites, iluminación, rotaciones, referencias y efectos. No debe competir con Blender ni incluir un pipeline PBR completo.

- Importación de OBJ/GLTF en una fase avanzada.
- Cámara ortográfica y perspectiva.
- Luces simples, sombras opcionales y fondo transparente.
- Render a resolución baja con pixel snapping.
- Conversión de secuencias 3D a spritesheet o asset animado.

9. Evaluación y caché

- Hash por nodo basado en entradas, parámetros, versión y seed.

- Caché en memoria y disco con limpieza configurable.
- Cancelación cooperativa y prioridad del nodo visible.
- Modo preview de baja resolución y modo final.
- Profiler por nodo: tiempo, memoria y recálculos.
- Errores propagados con ruta completa, sin cerrar el proyecto.

10. Colecciones y subgrafos

- Convertir una selección en grupo con entradas y salidas públicas.
- Guardar grupo como colección reutilizable.
- Versionar colecciones y avisar incompatibilidades.
- Crear instancias enlazadas o copias independientes.
- Publicar presets de parámetros sin duplicar el grafo.

11. Integraciones

- Entradas vivas desde Assets o documentos Pixel Art.
- Publicar salida como nuevo asset o revisión derivada.
- Usar salida animada como VFX, fondo o tile en Mapas.
- Ejecutar variaciones masivas y simulaciones mediante Jobs.
- Exportar el grafo horneado; en Morph, conservar opcionalmente una representación procedural compatible.

12. Fases

Fase	Alcance
P0	Grafo, puertos tipados, previews, input/output, transform, color, blend y caché.
P1	Arrays, grupos, colecciones, seeds, publicación como asset y procesamiento por lotes.
P2	Timeline, keyframes, curvas, expresiones y exportación animada.
P3	Feedback, partículas y simulaciones 2D básicas.
P4	Fluidos, humo, rigidbody, verlet y 3D de apoyo.

13. Criterios

- [] Un grafo vuelve a producir el mismo resultado con las mismas entradas y seed.
- [] Modificar un nodo recalcula solo dependientes.
- [] Cada nodo puede mostrar preview o error.
- [] Los grafos pueden agruparse y reutilizarse.
- [] El usuario puede cancelar evaluaciones largas.
- [] Las salidas se publican como assets con trazabilidad.
- [] El sistema conserva animación y no introduce antialiasing involuntario.

MOSAICO

Pestaña Mapas

Editor de niveles 2D con modo manual y generación pattern-based mediante reglas

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito
02	Dos modos complementarios
03	Documento de mapa
04	Interfaz
05	Tilesheets y paletas de entrada
06	Herramientas manuales
07	Sistema de reglas
08	Editor de reglas
09	Evaluación y regeneración
10	Capas, overrides y horneado
11	Objetos y colisiones
12	Propiedades
13	Validación y diagnóstico
14	Integración y exportación
15	Fases
16	Criterios de aceptación

1. Propósito

Mapas permite construir niveles 2D usando tilesets y assets del proyecto. Su identidad combina dos formas de trabajo dentro de la misma pestaña: edición manual precisa y creación basada en reglas inspirada en TileKit. El usuario puede colocar tiles directamente o dibujar un mapa básico de entrada para que un ruleset produzca automáticamente el mapa visual final.

Decisión central

La generación por reglas no pertenece a Pipelines ni a una pestaña futura. Es una capacidad principal de Mapas. Pipelines genera y transforma imágenes; Mapas interpreta patrones espaciales de una grilla para elegir tiles, stamps, objetos y capas.

2. Dos modos complementarios

Modo	Qué edita el usuario	Resultado
Manual	Tiles, stamps, objetos, colisiones y propiedades finales.	Mapa final controlado celda por celda.
Reglas	Mapa de entrada con tiles básicos/semánticos y un conjunto de reglas.	Mapa de salida derivado, regenerable y reproducible.
Híbrido	Base generada + capas de override manual.	Resultado reglado con correcciones que no se pierden al regenerar.

El cambio de modo no crea documentos separados. Un mismo mapa puede contener capas manuales, capas fuente de reglas, resultados derivados y overrides. El usuario debe saber siempre qué capa está editando y qué contenido será recalculado.

3. Documento de mapa

Dato	Descripción
Grilla	Tamaño de celda, ancho, alto, origen, orientación y unidades.
Tilesets	Assets preparados, índices, animaciones, tags, pivotes y variantes.
Paleta de entrada	Tiles básicos o tokens como vacío, suelo, pared, agua, camino, altura o bioma.
Rulesets	Patrones, condiciones, salidas, prioridad, transformaciones, seed y versión.
Capas fuente	Grillas editables que expresan la intención o estructura básica.
Capas derivadas	Tiles y objetos producidos por el motor de reglas; no se editan destructivamente.
Overrides	Correcciones manuales persistentes aplicadas encima del resultado generado.
Capas manuales	Tile, objeto, imagen, colisión, navegación, marcador y metadata.
Propiedades	Clave/valor a nivel de mapa, capa, celda, región u objeto.

4. Interfaz

- Canvas central con zoom, pan, grid, límites y preview pixel-perfect.
- Selector visible de modo: Manual, Reglas o Resultado/Preview.
- Panel de tiles finales, objetos y stamps para edición manual.
- Panel de tokens básicos para pintar el mapa de entrada.
- Panel de capas que distingue fuente, derivada, override y manual mediante iconos y etiquetas.
- Panel de rulesets con reglas ordenables, activación, grupos, búsqueda y errores.
- Inspector contextual de celda: token de entrada, patrón reconocido, regla ganadora y salida.

- Vista dividida opcional: entrada a la izquierda y resultado a la derecha.
- Minimapa, información de coordenada y contador de celdas afectadas.

5. Tilesheets y paletas de entrada

- Importación desde Assets o mediante un asistente que cree el asset correspondiente.
- Tamaño de tile, margen, spacing, offset y preview numerado.
- Selección de subconjunto, nombres, tags y categorías por tile.
- Soporte de tiles animados, variantes y stamps multitie.
- Pivote, colisión sugerida y metadata por tile.
- La paleta de entrada debe usar tokens simples y claramente distinguibles del arte final.
- Los tokens pueden ser tiles básicos dibujados por el usuario o categorías semánticas reutilizables.
- Un ruleset puede declarar qué paletas de entrada y tilesets finales acepta.

6. Herramientas manuales

Herramienta	Comportamiento
Lápiz	Pinta tile, token o patrón seleccionado; arrastre continuo sin huecos.
Borrador	Elimina celdas u objetos según la capa activa.
Cuentagotas	Toma tile, token, objeto o propiedad de una celda.
Línea	Traza tiles o tokens entre dos puntos con preview.
Rectángulo	Contorno o relleno; actualiza reglas en la zona afectada.
Círculo/elipse	Contorno o relleno discretizado en grilla.
Balde	Rellena una región contigua con límites, preview y cancelación.
Selección	Mover, copiar, pegar, voltear y rotar 90° cuando el tipo lo permita.
Stamp/brush	Patrones multitie, habitaciones, estructuras o decoraciones guardadas.
Bloqueo/override	Protege una celda generada o crea una corrección manual persistente.

7. Sistema de reglas pattern-based

El motor toma una o varias capas de entrada compuestas por tokens básicos y busca patrones definidos por el usuario. Cuando una regla coincide, coloca una salida del tileset, un stamp, una variante, un objeto o una modificación de capa. El resultado se conserva como dato derivado y puede regenerarse.

Elemento de regla	Especificación
Patrón de entrada	Matriz 1x1, 3x3, 5x5 o tamaño impar configurable alrededor de una celda ancla.
Condiciones	Token exacto, categoría/tag, cualquiera, vacío, no es, igual al centro o pertenece a conjunto.
Capas consultadas	Una o varias capas fuente: terreno, altura, bioma, caminos u otras definidas por el usuario.
Salida	Tile, variante, stamp multitie, objeto, marker, propiedad, borrar o no modificar.
Destino	Capa derivada concreta y offset respecto al ancla.
Prioridad	Orden explícito y especificidad; una regla más concreta puede vencer a una general.
Transformaciones	Rotaciones y reflejos opcionales para crear variantes sin duplicar reglas.
Variación	Salidas ponderadas con seed estable por mapa, regla y coordenada.
Iteración	Una pasada, múltiples pasadas limitadas o hasta estabilidad con protección contra ciclos.

Elemento de regla	Especificación
Ámbito	Ruleset, región, capa, selección o operación de pincel.

Auto-tiling clásico incluido

Las máscaras 4-way y 8-way no son un sistema separado: deben existir como presets y asistentes capaces de generar reglas pattern-based para centros, bordes, esquinas internas, esquinas externas y transiciones.

8. Editor de reglas

- Lista ordenada de reglas con nombre, grupo, prioridad, activación y contador de coincidencias.
- Editor visual de patrón en cuadrícula, con celda ancla claramente marcada.
- Paleta de condiciones: exacto, categoría, cualquiera, vacío, negación, mismo que centro y conjunto.
- Selector visual de salida desde el tileset, stamp, objeto o propiedad.
- Controles para rotación, reflexión, peso aleatorio, offset, capa destino y número de pasada.
- Botón “crear regla desde selección” que captura entrada y salida de una zona de ejemplo.
- Sandbox de prueba pequeño para editar entradas y ver el resultado al instante.
- Duplicar, agrupar, habilitar/deshabilitar, importar, exportar y versionar rulesets.
- Detector de reglas imposibles, duplicadas, ocultas por prioridad o potencialmente cíclicas.

9. Evaluación y regeneración

- Al pintar una celda fuente, recalcular únicamente la zona cuyo patrón pueda verse afectado.
- Agrupar una pincelada completa en una sola operación de undo/redo y una sola regeneración incremental.
- Mostrar preview antes de confirmar rellenos o transformaciones grandes.
- Mantener seed y orden de reglas para obtener resultados reproducibles.
- Permitir regenerar selección, capa, región o mapa completo.
- Enviar regeneraciones grandes a Jobs sin congelar el editor.
- Modo paso a paso para visualizar cada pasada y detectar reglas conflictivas.
- Límite de iteraciones, detección de oscilación y cancelación segura.
- Caché por chunks o regiones para mapas extensos.

10. Capas, overrides y horneado

Tipo de capa	Regla de edición
Fuente	Editable; contiene tokens básicos y dispara regeneración.
Derivada	Generada por ruleset; lectura/inspección, no pintura destructiva directa.
Override	Correcciones manuales persistentes que prevalecen sobre la capa derivada.
Manual	Contenido final independiente de reglas.
Bake	Copia congelada y editable del resultado, desligada o vinculada a una revisión concreta.

- Los overrides deben marcar visualmente que ocultan una salida generada.
- Regenerar nunca debe borrar overrides sin confirmación explícita.
- El usuario puede limpiar overrides por selección o convertirlos en nuevas reglas.
- Hornear crea un snapshot; debe advertir si se perderá la capacidad de regeneración.
- Puede conservarse simultáneamente el ruleset y un resultado horneado para exportación.

11. Objetos y colisiones

- Colocar sprites, animaciones, VFX y marcadores como instancias manuales.
- Permitir que reglas coloquen decoraciones, puertas, árboles, luces, spawns o triggers neutrales.
- Evitar duplicados mediante claves de generación estables por regla y coordenada.
- Posición en píxeles o snap a grilla; escala entera por defecto.
- Pivote, rotación limitada y z-order.

- Formas de colisión: rectángulo, círculo, polígono y por tile.
- Las colisiones pueden provenir del tile, de una regla o de una capa manual de override.
- Capas de navegación y zonas de coste como metadatos neutrales.

12. Propiedades

Nivel	Ejemplos
Mapa	música, clima, nombre, bioma, tamaño lógico, seed y ruleset activo.
Capa	grupo, función fuente/derivada, parallax, colisión y visibilidad inicial.
Región	bioma, conjunto de reglas, prioridad, dificultad o estilo visual.
Objeto	tipo, ID de spawn, diálogo, loot, script sugerido y procedencia de regla.
Celda	token, terreno, daño, sonido, velocidad, altura y regla aplicada.

13. Validación y diagnóstico

- Referencias rotas o tilesets/rulesets desactualizados.
- Tokens de entrada sin regla aplicable y celdas de salida vacías inesperadas.
- Reglas duplicadas, inalcanzables, ambiguas o siempre ocultadas por otra prioridad.
- Ciclos, oscilaciones o exceso de iteraciones.
- Stamps fuera de límites, solapamientos y objetos duplicados.
- Overrides huérfanos tras cambiar el tamaño o estructura del mapa.
- Inspector de celda con patrón leído, reglas candidatas, regla ganadora, seed y salida.
- Mapa de calor opcional de coincidencias y zonas sin cobertura.
- Errores que bloquean exportación separados de advertencias informativas.

14. Integración y exportación

- Recibir tilesets, stamps y objetos desde Assets.
- Usar VFX o fondos generados en Pipelines, sin trasladar allí el motor de reglas de tilemap.
- Publicar el mapa final y su estructura en Mundo.
- Enviar regeneración, bake, thumbnails y validaciones masivas a Jobs.
- Godot, Unity y Unreal reciben por defecto el resultado horneado y los datos finales.
- Morph puede recibir además capas fuente, rulesets, procedencia y semántica cuando su contrato lo permita.
- Permitir exportar/importar rulesets como recursos reutilizables de Mosaico.
- No incluir lógica específica del motor dentro del editor de Mapas.

15. Fases

Fase	Alcance
M0 - Manual	Canvas editable, tilesheets, herramientas, capas, selección, objetos, guardado y undo/redo.
M1 - Auto-tiling	Máscaras 4/8-way, presets, integración con lápiz/formas/balde y diagnóstico básico.
M2 - Reglas	Mapa fuente, rulesets, editor visual 3x3/5x5, prioridades, transformaciones y salida derivada.
M3 - Híbrido	Overrides, bake, variaciones ponderadas, reglas multicapas, stamps y objetos.
M4 - Avanzado	Multi-pass, regiones, creación desde ejemplos, análisis de cobertura y ejecución por Jobs.
M5 - Procedural	Pinceles de intención, restricciones de alto nivel y WFC como complemento posterior, no sustituto.

16. Criterios de aceptación

- [] El modo manual funciona sin depender de reglas.
- [] Pintar tokens con lápiz, línea, rectángulo, círculo o balde actualiza correctamente el resultado.
- [] Las mismas entradas, reglas, versión y seed producen el mismo mapa.
- [] Modificar una celda recalcula solo la vecindad necesaria.
- [] El usuario puede identificar qué regla produjo cualquier tile u objeto.
- [] Regenerar no destruye overrides manuales.
- [] Las reglas pueden guardarse, reutilizarse y versionarse.
- [] Los ciclos y conflictos se detectan sin bloquear ni corromper el mapa.
- [] Los objetos mantienen IDs estables entre regeneraciones cuando su regla y ancla no cambian.
- [] El mapa se registra en Mundo sin duplicarse.
- [] La exportación final conserva capas, propiedades, colisiones y trazabilidad según el perfil.

MOSAICO

Pestaña Mundo

Organización macro de regiones, mapas, conexiones y reglas globales

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito reconstruido
02	Qué representa un mundo
03	Interfaz
04	Regiones
05	Conexiones
06	Coordenadas
07	Capas globales
08	Instancias y plantillas
09	Validación
10	Integración
11	Fases
12	Criterios

1. Propósito reconstruido

Mundo debe resolver una escala que Mapas no cubre: organizar muchos niveles como un conjunto navegable y exportable. Es el equivalente a una vista macro del juego, donde se definen regiones, conexiones, coordenadas globales, agrupaciones y propiedades compartidas.

Por qué esta definición

Dentro de las siete pestañas, esta función evita duplicar Mapas y da sentido a Exportar: el motor recibe no solo niveles aislados, sino una estructura mundial coherente.

Límite

Mundo no es un editor de gameplay, misiones, NPCs ni simulación social. Puede almacenar referencias y metadatos, pero la lógica se implementa en el motor de destino.

2. Qué representa un mundo

Entidad	Función
World	Contenedor raíz con ajustes globales y uno o más espacios.
Region	Agrupar mapas por bioma, capítulo, zona, planeta, dungeon o criterio libre.
Map Node	Referencia a un documento de Mapas, con posición y estado.
Connection	Portal, puerta, borde, viaje, ascensor, teletransporte o transición.
Global Layer	Información que atraviesa mapas: clima, altura, música, facción, navegación macro.
Streaming Group	Conjunto de mapas que el motor puede cargar juntos.
World Marker	Spawn global, destino, punto de interés o referencia narrativa.

3. Interfaz

- Canvas macro con nodos de mapa, regiones, conexiones y grupos.
- Modo espacial: coloca mapas a escala en coordenadas globales.
- Modo grafo: prioriza conexiones y flujo, sin depender de distancia física.
- Árbol lateral de mundos, regiones, mapas y marcadores.
- Inspector de selección con propiedades y referencias.
- Panel de problemas para conexiones incompletas o mapas faltantes.
- Mini preview del mapa seleccionado y botón para abrirlo en Mapas.

4. Regiones

- Nombre, color UI, tipo, tags y descripción.
- Propiedades heredables: bioma, clima, música, escala, iluminación o reglas de exportación.
- Regiones anidadas opcionales en una fase posterior.
- Contención visual y lógica; mover una región puede mover sus mapas en modo espacial.
- Filtros y exportación parcial por región.

5. Conexiones

Tipo	Datos mínimos
Borde	Mapa origen, lado/zona, mapa destino, lado/zona y offset.
Portal/puerta	Marcador origen, destino, transición y condiciones neutrales.

Tipo	Datos mínimos
Viaje	Nodo origen/destino, coste o etiqueta y escena de transición.
Unidireccional	Dirección permitida y destino de retorno opcional.
Instancia	Mapa plantilla, ID de instancia y parámetros.

- Cada extremo debe usar IDs estables de mapa y marcador.
- El editor puede crear automáticamente el extremo inverso cuando sea simétrico.
- Cambiar o borrar un marcador debe mostrar todas las conexiones afectadas.
- La transición visual se guarda como metadato neutral, no como código del motor.

6. Coordenadas

- Coordenada global opcional por mapa y región.
- Escala lógica configurable: tiles, píxeles, metros neutrales o unidades del proyecto.
- Snap de mapas para mundos continuos.
- Detección de superposición y huecos cuando se declara continuidad.
- Origen mundial y transform de exportación por perfil.

7. Capas globales

Capa	Uso
Biomás	Asignar zonas y propiedades ambientales.
Altura	Ordenar pisos, cuevas, subsuelo o niveles verticales.
Clima	Referencias a presets o Pipelines visuales.
Audio	Música o ambientes sugeridos por región.
Navegación macro	Rutas entre mapas, costes y restricciones.
Streaming	Agrupaciones y prioridades de carga.
Narrativa neutral	Capítulo, acto, estado requerido o etiqueta; sin ejecutar lógica.

8. Instancias y plantillas

- Un mapa puede usarse como plantilla de varias instancias.
- Cada instancia conserva un ID y overrides de propiedades permitidas.
- Los cambios estructurales del mapa plantilla se propagan con revisión.
- Los overrides incompatibles se marcan para resolver.
- Útil para casas repetidas, habitaciones, arenas o pisos procedurales.

9. Validación

- Mapas faltantes o desactualizados.
- Conexiones sin destino, con marcadores inexistentes o tipos incompatibles.
- IDs globales duplicados.
- Mapas no alcanzables desde el inicio cuando se activa esa regla.
- Regiones vacías, superposiciones o coordenadas inconsistentes.
- Dependencias no incluidas en el perfil de exportación.

10. Integración

- Añadir mapas existentes o crear uno nuevo desde la pestaña.
- Usar assets y previews para marcadores globales.
- Referenciar Pipelines como presets de clima o fondo, sin ejecutar simulación dentro de Mundo.
- Enviar generación de miniaturas, validación y builds parciales a Jobs.
- Entregar a Exportar la estructura mundial neutral y sus dependencias.

11. Fases

Fase	Alcance
W0	Grafo de mapas, regiones, conexiones, propiedades y validación básica.
W1	Modo espacial, coordenadas, bordes continuos y preview de mapas.
W2	Capas globales, streaming groups, instancias y exportación parcial.
W3	Herramientas de análisis: alcanzabilidad, rutas, cobertura y coherencia.

12. Criterios

- [] Un mundo puede contener y abrir múltiples mapas sin duplicarlos.
- [] Las conexiones sobreviven a renombrados y movimientos.
- [] Se detectan referencias rotas antes de exportar.
- [] Modo grafo y modo espacial representan los mismos datos.
- [] Las regiones pueden exportarse de forma parcial.
- [] Mapas y Mundo no almacenan copias divergentes del mismo contenido.
- [] El formato neutral puede adaptarse a varios motores.

MOSAICO

Pestaña Jobs

Centro de ejecución, procesamiento por lotes, caché y seguimiento de tareas

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito reconstruido
02	Qué es un job
03	Interfaz
04	Estados
05	Fuentes de jobs
06	Cola y recursos
07	Logs y resultados
08	Repetición
09	Seguridad
10	Integración
11	Criterios

1. Propósito reconstruido

Jobs es el centro operativo de Mosaico. Recibe tareas que pueden tardar, repetirse o procesar muchos archivos: evaluar Pipelines, generar atlas, convertir assets, hornear simulaciones, validar mundos y construir exportaciones.

Separación clave

Pipelines define una receta; Jobs crea una ejecución concreta de esa receta con entradas, parámetros, prioridad, progreso, logs y resultados.

2. Qué es un job

Campo	Descripción
ID	Identificador único de ejecución.
Tipo	Pipeline, importación, conversión, atlas, simulación, validación, exportación u otro.
Origen	Pestaña y documento que lo creó.
Entradas	Assets, mapas, mundo, perfiles o archivos.
Configuración	Parámetros congelados para reproducir la ejecución.
Estado	Pendiente, preparando, ejecutando, pausado, cancelando, completado, fallido o inválido.
Progreso	Porcentaje, etapa, elemento actual y estimación interna opcional.
Resultado	Archivos, assets, caché, build, informe o errores.
Provenance	Versiones de nodos, schema, aplicación y hash de dependencias.

3. Interfaz

- Lista de trabajos activos, pendientes, completados y fallidos.
- Filtros por tipo, estado, proyecto, fecha, origen y prioridad.
- Panel de detalle con etapas, log estructurado, entradas y salidas.
- Acciones: pausar, reanudar, cancelar, reintentar, duplicar y abrir resultado.
- Indicadores de CPU, memoria, disco y workers utilizados.
- Notificación al terminar, configurable por severidad.

4. Estados

```

PENDING -> PREPARING -> RUNNING -> COMPLETED
                |               |
                |               +--> PAUSED -> RUNNING
                +-----> FAILED -> RETRY
RUNNING -> CANCELLING -> CANCELLED
ANY -> INVALIDATED cuando cambia una dependencia crítica
  
```

Cancelar debe detener de forma segura y limpiar archivos temporales. Pausar solo estará disponible para tipos de trabajo que puedan guardar checkpoint.

5. Fuentes de jobs

Origen	Ejemplos
Assets	Importar carpeta, crear miniaturas, convertir, reescalar, generar atlas.

Origen	Ejemplos
Pipelines	Evaluar lotes, hornear animación, simular fluidos, publicar variantes.
Mapas	Generar preview, validar lote, baking de capas o colisiones.
Mundo	Validar conexiones, crear atlas global, exportar región.
Exportar	Construir paquete, copiar, convertir formatos y generar manifiesto.
Sistema	Migrar schema, limpiar caché, reconstruir índices.

6. Cola y recursos

- Prioridad: interactivo, alto, normal y bajo.
- Límites de concurrencia globales y por tipo.
- Worker CPU como base; GPU opcional y backend identificado.
- No ejecutar dos jobs que escriban el mismo destino sin bloqueo.
- Espacio de disco estimado y advertencia antes de tareas grandes.
- Modo “usar pocos recursos” para trabajar mientras corren procesos.

7. Logs y resultados

- Logs estructurados por timestamp, etapa, severidad y entidad.
- Errores con acción recomendada y enlace para abrir el elemento afectado.
- Resultados atómicos: escribir en temporal y mover al finalizar.
- Comparación de resultados entre reintentos.
- Retención configurable de historial y limpieza automática segura.
- Exportar informe de job en JSON o texto.

8. Repetición

- Duplicar un job conserva configuración y permite cambiar entradas.
- Presets de job para tareas habituales.
- Watch folders opcionales en una fase posterior.
- Ejecución manual por defecto; evitar automatizaciones ocultas que modifiquen el proyecto.
- Reintento automático limitado solo para fallos transitorios identificados.

9. Seguridad y consistencia

- Snapshot de parámetros y dependencias al iniciar.
- Marcar resultado como obsoleto cuando cambian las fuentes.
- Bloqueo de rutas críticas y copia previa para exportaciones incrementales.
- Sandbox lógico para nodos o scripts de terceros.
- Límites de tiempo, memoria y tamaño configurables.
- Recuperación de jobs interrumpidos tras reiniciar la aplicación cuando sea viable.

10. Integración

Cada pestaña puede crear jobs mediante una acción contextual, pero Jobs es el único lugar que controla ejecución, recursos, historial y reintentos. Abrir un job desde su origen debe seleccionar el documento o nodo que lo generó.

11. Criterios

- [] Un job largo no congela la interfaz.
- [] Cancelar no deja resultados parciales como válidos.
- [] Los logs permiten identificar el asset, nodo, mapa o archivo causante.
- [] Un job puede reintentarse con la misma configuración.
- [] Los resultados conservan trazabilidad y estado de obsolescencia.

-
- [] La cola respeta prioridades y límites.
 - [] La aplicación recupera historial útil tras reiniciarse.

MOSAICO

Pestaña Exportar

Perfiles, validación y adaptadores para Godot, Unity, Unreal y Morph

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Propósito
02	Formato neutral
03	Interfaz
04	Perfiles
05	Validación
06	Godot
07	Unity
08	Unreal
09	Morph
10	Reglas de mapas
11	Builds y manifiestos
12	Actualización incremental
13	Criterios

1. Propósito

Exportar es la última etapa de Mosaico. Reúne assets, mapas, mundos y resultados horneados, valida dependencias y construye un paquete adaptado al motor de destino. No debe contaminar las pestañas de creación con detalles específicos de cada motor.

Arquitectura

Todas las pestañas producen un formato neutral. Cada adaptador traduce ese formato a Godot, Unity, Unreal o Morph.

2. Formato neutral

Componente	Datos
Manifest	Proyecto, versión, fecha, perfil, archivos, hashes y dependencias.
Assets	Imágenes, atlas, clips, pivotes, tags y configuración neutral.
Maps	Grilla, capas, tiles, objetos, colisiones, propiedades y referencias.
Map rules	Paletas de entrada, rulesets, versiones, seeds, overrides y procedencia de resultados.
World	Regiones, conexiones, coordenadas, markers y streaming groups.
VFX	Secuencias horneadas, spritesheets, parámetros o grafo compatible.
Metadata	Campos neutrales y extensiones namespaced por motor.

3. Interfaz

- Selector de perfil de exportación.
- Árbol de contenido incluido y excluido.
- Selector por mapa: exportar resultado horneado, fuente + reglas o ambos cuando el destino lo soporte.
- Resumen de tamaño, cantidad de archivos y problemas.
- Preview de rutas de destino y nombres generados.
- Validación previa con errores bloqueantes y advertencias.
- Botón construir que crea un Job y muestra progreso.
- Historial de builds con diff respecto al anterior.

4. Perfiles

Campo	Ejemplos
Motor	Godot, Unity, Unreal o Morph.
Versión	Versión objetivo y compatibilidad del adaptador.
Destino	Carpeta nueva, paquete o proyecto existente.
Escala	Pixels per unit, tamaño de tile y factor de exportación.
Texturas	Filtro, compresión, mipmaps, alpha y límites.
Nombres	Convenciones de carpetas, archivos y recursos.
Contenido	Mundos, regiones, mapas y colecciones incluidos.
Reglas	Solo bake, incluir fuentes/rulesets o convertir subconjunto compatible.
Modo	Build limpio, incremental o paquete portable.

Campo

Ejemplos

5. Validación

- Assets o mapas faltantes y referencias rotas.
- Pivotes, tamaños o tiles incompatibles con el perfil.
- Nombres no válidos o colisiones de ruta.
- Funciones neutrales sin equivalente en el motor.
- Pipelines sin hornear cuando el adaptador no soporta el grafo.
- Mapas reglados sin resultado actualizado o con reglas en error.
- Conexiones de Mundo incompletas.
- Archivos modificados externamente en un proyecto destino.

6. Adaptador Godot

- Generar PNG/atlas y recursos de sprites/animaciones compatibles.
- Crear TileSet y TileMap/TileMapLayer según la versión objetivo.
- Traducir colisiones, propiedades y objetos a escenas o recursos.
- Exportar el resultado horneado de reglas como capas finales.
- Opcionalmente conservar tokens y reglas como recursos auxiliares para un addon Mosaico, sin exigirlo al runtime.
- Crear escenas por mapa y un recurso/escena para Mundo.
- Import settings con nearest filter y mipmaps desactivados por defecto para pixel art.

7. Adaptador Unity

- Configurar TextureImporter: Point, alpha, pixels per unit, compression y mipmaps.
- Generar sprites múltiples, AnimationClips y Animator opcional.
- Crear TileBase/Tilemap y prefabs para objetos.
- Exportar la salida reglada ya resuelta; conversión futura de reglas solo mediante paquete/importer controlado.
- Serializar propiedades neutrales en ScriptableObjects o componentes del addon.
- Crear escenas por mapa y estructura mundial mediante assets de datos.
- Opcional: paquete UPM o editor importer para sincronización.

8. Adaptador Unreal

- Orientado inicialmente a Paper2D: sprites, flipbooks, tile sets y tile maps.
- Configurar filtros, grupos de textura, compresión y unidades.
- Generar datos de colisión y propiedades en assets auxiliares.
- Exportar mapas reglados como resultado horneado y manifiesto de procedencia.
- Mapas exportados como datos y, cuando sea seguro, assets mediante plugin/importer.
- Mundo traducido a data assets, niveles o world partition según alcance y versión.
- Preferir un plugin de importación frente a escribir binarios propietarios directamente.

9. Adaptador Morph

Morph es el destino con integración más profunda porque su filosofía AI-first puede aprovechar la semántica de Mosaico. El adaptador debe conservar IDs, roles, intención, relaciones, reglas y procedencia, no solo imágenes finales.

- Exportar assets con metadatos semánticos y referencias estables.
- Traducir mapas a entidades/componentes, capas, colisiones y objetos del engine.
- Adjuntar mapas fuente, tokens, rulesets, seeds, overrides y relación source -> output.
- Traducir Mundo a grafo de escenas, regiones, portales y streaming.
- Conservar manifiestos legibles por agentes y MCP.
- Adjuntar trazas de fuente: documento Pixel Art, asset, Pipeline, regla y revisión.
- Permitir hot-reload o sync incremental cuando Morph exponga el contrato necesario.
- Exportar Pipelines horneados inicialmente; posteriormente, convertir subconjuntos compatibles a operaciones runtime.

10. Reglas de mapas

Destino	Política inicial
Godot	Bake obligatorio para compatibilidad universal; metadata de ruleset opcional mediante addon.
Unity	Bake obligatorio; posible conversión futura a RuleTile solo cuando no cambie el significado.
Unreal	Bake obligatorio; ruleset incluido como JSON/manifest opcional para herramientas.
Morph	Bake + fuente + ruleset + procedencia; conversión procedural cuando exista contrato estable.

No traducir por aproximación

Las reglas de Mosaico pueden ser más expresivas que el auto-tiling de un motor. Cuando no exista equivalencia exacta, Exportar debe hornear el resultado y advertirlo, no inventar una traducción incompleta.

11. Builds y manifiestos

- Cada build recibe ID, timestamp, perfil, versión y hash.
- Manifiesto de archivos con origen, dependencia, ruleset y revisión usada.
- Informe de validación y conversiones realizadas.
- Carpeta staging separada del destino final.
- Opción de empaquetar ZIP o copiar a proyecto.
- Build reproducible con la misma revisión, seed y perfil.

12. Actualización incremental

- Comparar hashes y copiar solo cambios.
- Regenerar únicamente mapas cuyos inputs, rulesets o dependencias cambiaron.
- No borrar archivos desconocidos sin una lista de propiedad del adaptador.
- Preview de operaciones: crear, actualizar, omitir y eliminar.
- Backup o control de versiones recomendado antes de modificar proyecto existente.
- Detectar modificaciones externas y pedir resolución, no sobrescribir silenciosamente.

13. Criterios

- [] Un mismo proyecto neutral puede usar varios perfiles.
- [] Los adaptadores no modifican fuentes de Mosaico.
- [] La validación ocurre antes de escribir en destino.
- [] Cada build produce manifiesto e informe.
- [] La exportación incremental no borra archivos ajenos.
- [] Godot, Unity y Unreal reciben pixel art sin filtrado accidental.
- [] Los mapas reglados se exportan de forma reproducible y sin resultados obsoletos.
- [] Morph conserva semántica, reglas y trazabilidad además de archivos visuales.

MOSAICO

Hoja de ruta e integración

Orden de implementación, dependencias y definición de terminado para las siete pestañas

Documento de alcance, experiencia de usuario, datos, flujos, validaciones y criterios de aceptación.

Este documento define qué debe hacer Mosaico y cómo deben relacionarse sus módulos. No describe una implementación cerrada: establece el comportamiento esperado para que el desarrollo pueda dividirse entre agentes sin perder coherencia.

Contenido

N.º	Sección
01	Principio de entrega
02	Estado actual
03	Fases
04	Dependencias
05	MVP por pestaña
06	Paquetes para agentes
07	Pruebas
08	Riesgos
09	Definición de terminado

1. Principio de entrega

Mosaico debe crecer mediante cortes verticales utilizables, no mediante siete pestañas parcialmente vacías. Cada fase debe producir un flujo que se pueda probar con un proyecto real.

Orden recomendado

Consolidar Pixel Art y Assets; completar el modo manual de Mapas; añadir auto-tiling como preset; implementar después el núcleo pattern-based de reglas; construir Pipelines, Mundo y Jobs; finalmente cerrar Exportar con un adaptador completo.

2. Estado actual

Pestaña	Estado indicado	Siguiente objetivo
Pixel Art	Funcional en desarrollo.	Cerrar herramientas, capas, animación, guardado y contratos con Assets.
Assets	Funcional en desarrollo.	Modelo estable, slicing, biblioteca y trazabilidad.
Mapas	Funcional en desarrollo.	Cerrar modo manual y diseñar desde ahora el modelo de capas fuente, rulesets, resultados y overrides.
Pipelines	Existe como pestaña/concepto.	Reorientar a compositor procedural tipo Pixel Composer.
Mundo	Función reconstruida.	Implementar grafo de mapas y regiones.
Jobs	Función reconstruida.	Implementar runner compartido para tareas largas.
Exportar	Concepto recordado.	Formato neutral y primer adaptador de motor.

3. Fases

Fase	Resultado verificable
F0 - Core	Proyecto versionado, IDs, referencias, eventos, undo/redo, caché y recuperación.
F1 - Autoría	Pixel Art + Assets producen sprites, tilesets y animaciones trazables.
F2 - Mapa manual	Mapas pinta tiles/objetos, maneja capas, selección, guardado y validación.
F3 - Auto-tiling	Presets 4/8-way funcionan con lápiz, formas y balde sobre el editor manual.
F4 - Reglas TileKit-like	Mapa de entrada + ruleset produce salida derivada reproducible con diagnóstico.
F5 - Export vertical	Un mapa manual o reglado se hornea y exporta correctamente a Godot o Morph.
F6 - Pipelines	Grafo básico genera y publica assets visuales no destructivos.
F7 - Mundo	Varios mapas se conectan, validan y exportan como estructura.
F8 - Jobs	Regeneraciones, lotes, simulaciones y builds no bloquean UI.
F9 - Adaptadores	Unity y Unreal se añaden sobre el formato neutral ya estable.
F10 - Avanzado	Reglas multicapas/multi-pass, partículas, simulación, 3D y WFC complementario.

4. Dependencias

```
Core Project
|-- Pixel Art -> Assets
|-- Assets -> Mapas Manual -> Rule Engine -> Resultado -> Mundo
|-- Assets -> Pipelines -> Assets
|-- Jobs sirve a Rules/Pipelines/Mapas/Mundo/Exportar
+++ Exportar consume Assets + Mapas + Rulesets/Bakes + Mundo
```

- El motor de reglas depende de un formato de grilla, capas, tilesets e IDs ya estable.
- Los overrides dependen de una separación clara entre fuente, derivado y manual.
- Mundo depende de un formato de mapa estable.
- Jobs depende de contratos cancelables y serializables.
- Exportar depende de IDs, manifiestos, seeds, rulesets y validadores.
- Simulación avanzada depende de Jobs y caché.
- Morph debe recibir semántica, por lo que depende del modelo de proyecto y metadata.

5. MVP por pestaña

Pestaña	MVP que sí entrega valor
Pixel Art	Dibujo, capas, frames, paletas, PNG/spritesheet y vínculo a Assets.
Assets	Biblioteca, importación, slicing, pivotes, tags y atlas simple.
Pipelines	Grafo tipado, 25-40 nodos esenciales, preview, caché y publicar salida.
Mapas	Modo manual completo + fuente básica + reglas 3x3 + prioridades + resultado derivado + overrides.
Mundo	Regiones, nodos de mapa, conexiones y exportación neutral.
Jobs	Cola, progreso, cancelar, reintentar, logs y resultados.
Exportar	Formato neutral + un adaptador completo + manifiesto + bake de mapas reglados.

6. Paquetes para agentes

- Cada paquete debe declarar archivos permitidos, contratos usados y exclusiones.
- Un agente no debe crear un segundo modelo de Asset, Map, Ruleset o Job.
- El motor de reglas debe desarrollarse separado de la UI y probarse con fixtures de grillas pequeñas.
- Cambios de schema requieren migración y fixtures.
- Las tareas visuales deben incluir capturas o golden files.
- Las tareas de exportación deben probarse en un proyecto mínimo del motor destino.
- Las tareas de Pipelines deben incluir grafo de prueba y resultado esperado.
- Las tareas de reglas deben incluir entrada, ruleset, seed, salida esperada y diagnóstico esperado.

7. Pruebas

Tipo	Cobertura
Unitarias	Operaciones raster, matching de patrones, prioridad, transformaciones, serialización y hashes.
Rule fixtures	Entrada + reglas + seed -> salida exacta; conflictos, vacíos, bordes, multi-pass y ciclos.
Round-trip	Guardar/abrir proyecto, asset, grafo, ruleset, mapa, mundo, job y perfil.
Integración	Enviar contenido entre pestañas y conservar IDs/procedencia.
Golden image	Resultados raster, atlas, tiles, mapas reglados, VFX y previews.
Export	Abrir paquete generado en motor objetivo y verificar importación.

Tipo	Cobertura
Stress	Mapas grandes, regeneración incremental, lotes, grafos pesados, cancelación y poca memoria.
Migración	Abrir versiones anteriores sin pérdida silenciosa.

8. Riesgos

Riesgo	Mitigación
Intentar copiar Pixel Composer completo	Fases estrictas y catálogo pequeño de nodos esenciales.
Confundir Pipelines con reglas de mapas	Motores separados: raster procedural frente a matching espacial discreto.
Reglas destruyen trabajo manual	Capas derivadas de solo lectura, overrides persistentes y bake explícito.
Editor de reglas demasiado complejo	Presets 4/8-way, creación desde ejemplos y sandbox visual.
Reglas no deterministas	Seeds, prioridad, orden y versiones explícitas.
Ciclos o regeneración lenta	Límites, diagnóstico, chunks, caché incremental y Jobs.
Mundo se vuelve motor	Limitarlo a estructura, referencias y metadatos.
Jobs se vuelve automatización opaca	Ejecución visible, manual por defecto y logs.
Exportadores frágiles	Formato neutral, bake seguro, addons/importers y manifiestos.
Rendimiento sin CUDA	CPU base, caché incremental y backends opcionales.
Pérdida de datos	Guardado atómico, backups, migraciones y recovery.

9. Definición de terminado

Criterios de aceptación

- [] La función cumple su documento de alcance o registra desviación aprobada.
- [] Guarda y reabre sin pérdida.
- [] Tiene estados vacío, carga, éxito, advertencia y error.
- [] Undo/redo o no-destructividad cubre las operaciones pertinentes.
- [] Los procesos largos pueden cancelarse.
- [] Los resultados reglados son deterministas y explicables por celda.
- [] Los overrides sobreviven a regeneraciones.
- [] Incluye pruebas y fixtures reales.
- [] No rompe IDs ni referencias.
- [] La UI mantiene coherencia con Pixel Art.
- [] La documentación técnica y de usuario está actualizada.
- [] QA visual confirma ausencia de clipping, antialiasing accidental y offsets fraccionarios.